Get started with Metal shader converter

Metal shader converter converts shader intermediate representations in LLVM IR bytecode into a form suitable to be loaded into Metal. Metal shader converter is available as both a library and a standalone executable. All the functionality exposed through the library interface is available via the standalone executable.

Overview

This document describes the IR conversion process, explains the binding model, synchronization considerations, and reflection capabilities, then provides general guidance and examples.

This document also presents `metal_irconverter_runtime.h`, a lightweight, header-only library that accompanies the Metal shader converter. `metal_irconverter_runtime.h` helps perform common tasks when working with pipeline states built from IR generated by the Metal shader converter.

Changelog

Version	Changes	Requirements
3	Support for function constants, framebuffer fetch, and the ability to compile intersection functions into Metal intersection function buffers. Default support for non-array texture types.	Metal intersection function buffers requires GPUFamily Apple9 or later, and targeting macOS/iOS 26, or later.
2.1	Bug fixes for specific cases of hull shaders	-
2	Support for shader debug information, globally-coherent memory access, and SV_CullPrimitive. Ability to compile ray generation shaders as compute kernels, and ray intersection functions as Metal intersection functions	Globally-coherent memory access requires targeting macOS 15, iOS 18, or later.
1.1	Support for ray tracing shaders.	Metal ray tracing support.
1	Initial release.	Argument buffers tier 2 support.

System requirements

Metal shader converter requires macOS 13 Ventura and Xcode 15, or later.

Metal shader converter for Windows requires Microsoft Windows 10 and Microsoft Visual Studio 2019, or later

Metal libraries built using Metal shader converter tools require a device that supports Argument Buffers Tier 2, running macOS 14 Sonoma, iOS 17 or later. If you build a Metal library for earlier OS versions, not all features will be supported.

Download Metal shader converter

Download Metal shader converter and Metal shader converter for Windows from https://developer.apple.com/downloads?q=Shader%20Converter. The installer contains the command-line tools, dynamic libraries, header files, code samples and documentation (installed under `/opt/metal-shaderconverter`).

Converting IR

To convert shaders from DXIL to Metal IR, you use Metal shader converter as a standalone executable (`metal-shaderconverter`) or as a dynamic library (`libmetalirconverter`). Metal shader converter and `libmetalirconverter` support both Windows and macOS.

STANDALONE EXECUTABLE

The Metal shader converter executable offers several options to customize code generation. In its most basic form, Metal shader converter takes a DXIL file as input and produces a metallib.

```
% metal-shaderconverter shader.dxil -o ./shader.metallib
```

By default, Metal shader converter generates metallib files that target the latest version of macOS at the time of release. You can inspect this version by running `metal-shaderconverter --version`. Make sure your Xcode is always up to date.

Run `metal-shaderconverter --help` to access all command-line options.

DYNAMIC LIBRARY

`libmetalirconverter` offers a C interface for easy integration into C, C++, Objective-C, and Swift codebases.

```
IRCompiler* pCompiler = IRCompilerCreate();
IRCompilerSetEntryPointName(pCompiler, "MainVS");

IRObject* pDXIL = IRObjectCreateFromDXIL(bytecode, size, IRBytecodeOwnershipNone);

// Compile DXIL to Metal IR:
IRError* pError = nullptr;
IRObject* pOutIR = IRCompilerAllocCompileAndLink(pCompiler, NULL, pDXIL, &pError);

if (!pOutIR)
{
  // Inspect pError to determine cause.
  IRErrorDestroy( pError );
}

// Retrieve Metallib:
IRMetalLibBinary* pMetallib = IRMetalLibBinaryCreate();
IRObjectGetMetalLibBinary(pOutIR, stage, pMetallib);
size_t metallibSize = IRMetalLibGetBytecodeSize(pMetallib);
uint8_t* metallib = new uint8_t[metallibSize];
IRMetalLibGetBytecode(pMetallib, metallib);

// Store the metallib to custom format or disk, or use to create a MTLLibrary.

delete [] metallib;
IRMetalLibBinaryDestroy(pMetallib);
IRObjectDestroy(pDXIL);
IRObjectDestroy(pOutIR);
IRCompilerDestroy(pCompiler);
```

You can use the library at runtime — as you develop your game, for example — in addition to more typical uses, such as asset conversion and packaging workflows.

Once your game runs on Metal, start converting your shaders ahead of time and directly distributing metallibs in your game.

CREATE A MTLLIBRARY INSTANCE FROM METALLIB BYTECODE

After you retrieve the metallib data and its corresponding size, you create a `MTLLibrary` via a `dispatch_data_t` object:

```
// Use metallib (at runtime):

NSError* __autoreleasing error = nil;
dispatch_data_t data =
    dispatch_data_create(metallib, metallibSize, dispatch_get_main_queue(), NULL);

id<MTLLibrary> lib = [device newLibraryWithData:data error:&error];

// lib and data are released by ARC.
```

Tip: On macOS, you can use function `IRMetalLibGetBytecodeData` to obtain a direct pointer into the metallib's bytecode, avoiding a copy operation.

MULTITHREADING CONSIDERATIONS

Metal shader converter supports multithreading the IR translation process, however, the `IRCompiler` instance isn't reentrant. Each thread in your program needs to create its own instance of `IRCompiler` to avoid race conditions. Once the compilation process completes, your program can reuse a compiler instance to convert further IR.

Binding model

THE TOP-LEVEL ARGUMENT BUFFER

In order to use the Metal IR, you bind global shader resources to your pipeline states via a "top-level" Argument Buffer. Metal shader converter offers two mechanisms to control the layout of the resource handles in the top-level Argument Buffer: an explicit mode via root signatures, and an automatic "linear" layout.

The top-level Argument Buffer is a resource shared between the CPU and GPU and, as such, you need to coordinate access to its memory to avoid race conditions. View the synchronization section of this document for best practices.

EXPLICIT LAYOUT VIA ROOT SIGNATURES

Use Root Signatures for maximum flexibility at resource binding time.

When you provide a root signature, or the input IR embeds one, Metal shader converter generates a layout for the top-level Argument Buffer that matches your specification. In particular, root signatures allow you to define the following resources in the top-level Argument Buffer:

- Inline constant data ("root constants") of arbitrary size.
- Pointers to Metal resources ("root arguments"). Pointers are 64-bit unsigned values corresponding to the `gpuAddress` or `resourceID` of the resource to reference. Note: textures need to always be placed in descriptor tables.
- Pointers to a resource table ("descriptor table"). In Metal you implement this table via Argument Buffers. Each entry in the table consists of three 64-bit unsigned values: a buffer GPU address, a texture handle, and flags. Use `IRDescriptorTableEntry` in the runtime companion header (described below) to correctly calculate offsets and cast pointer types.

After Metal shader converter generates its output IR, calculate the offsets of each resource in the top-level Argument Buffer by

taking the size of the root constants in bytes (if present), and adding the resource index multiplied by `sizeof(uint64_t)`, while observing that Metal shader converter adds padding to achieve the correct alignment for each resource type:

- Pointers: 8-byte alignment.
- 32-bit constants: 4-byte alignment.

When you use root signatures, you need to supply `SamplerState` objects in a descriptor table. Metal shader converter doesn't support providing samplers in the top-level argument buffer in this mode. Additionally, Metal shader converter doesn't support static samplers.

Use the explicit layout approach when porting root signatures or when your game uses bindless resources.

AUTOMATIC LINEAR LAYOUT

When you don't provide a root signature to the Metal shader converter at conversion time, and the input IR doesn't embed one, Metal shader converter automatically generates a linear layout of resources into the top-level Argument Buffer.

This is a simple layout where the top-level Argument Buffer directly references each resource through a small resource descriptor. Just like descriptor tables, each entry in the Argument Buffer consists of three `uint64_t` parameters.

If you use `libmetalirconverter`, you can reflect the Argument Buffer offsets using function:

```
void IRShaderReflectionGetResourceLocations(
                        IRShaderReflection* reflection,
                        IRResourceLocation* resourceLocations);
```

Alternatively, if you use Metal shader converter as a standalone tool, it conveniently writes resource locations into a reflection JSON file when you provide argument: `--output-reflection-file`.

The order in which Metal shader converter places your resources in the top-level Argument Buffer is implementation-specific. Always reference the offline reflection data to correctly determine resource offsets.

The runtime companion header, described later in this document, provides a helper struct and functions you can use to encode your resources into this Argument Buffer. These helpers start with the prefix: `IRDescriptorTable`.

Use the automatic layout mechanism when you don't need to produce a resource hierarchy and your game doesn't use bindless resources.

Because this mechanism avoids one level of indirection, it may provide a performance advantage compared to the explicit layout approach.

RESOURCE ENCODING

When you use root descriptors for your top-level Argument Buffer, you encode resource references into it by writing 64-bit GPU addresses (or Metal resource IDs) at their corresponding offsets within the Argument Buffer.

In addition to root resources, the top-level Argument Buffer may reference "descriptor tables" that you need to encode using a specific format.

This specific format also applies to resources the top-level Argument Buffer directly references when you generate an automatic linear layout of resources (i.e., when you do not provide a root signature to Metal shader converter).

The Metal shader converter companion header provides helper functions to help you encode resources into descriptor tables.

Use functions `IRDescriptorTableSetBuffer()`, `IRDescriptorTableSetTexture()`, and `IRDescriptorTableSetSampler()`, to encode resource references into descriptor tables, depending on the resource type to encode.

You may also implement this encoding yourself without using the runtime header. In this case, resource “descriptors” in descriptor tables always consist of three 64-bit unsigned ints representing the GPU address (for buffers and acceleration structures), the texture resource ID (for textures and sampler bindings), and metadata flags (64 bits) representing:

- For buffers:
  - The buffer length stored in the low 32 bits.
  - A texture buffer view offset stored in 2 bytes left-shifted 32 bits. Metal requires texture buffer views to be aligned to 16 bytes. This offset represents the padding necessary to achieve this alignment.
  - Whether the buffer is a typed buffer (`1`) or not (`0`) in the high bit.
- For textures:
  - The min LOD clamp in the low 32 bits.
  - 0 in the high 32 bits.
- For samplers, the LOD bias as a 32-bit floating point number.

TOP-LEVEL ARGUMENT BUFFER SYNCHRONIZATION

The top-level Argument Buffer is a shared resource that the CPU and GPU both may access simultaneously, and as such, you need to coordinate access to its memory to avoid race conditions.

In the Metal execution model, you first encode the work to perform into a command buffer, and then the GPU carries out your commands later, after you commit it. If each draw call modifies the top-level Argument Buffer as a shared resource, when the GPU executes your commands, only the last modification to the Argument Buffer is visible to the pipeline.

Furthermore, when your application handles multiple frames in flight, the CPU could overwrite memory locations the GPU is reading from.

To avoid these situations, you can provide each draw call with its own top-level Argument Buffer. While you can manage these as discrete or MTLHeap-based allocations, it would necessitate having one Argument Buffer per draw call and for each frame in flight. This can become challenging to manage and carry CPU overhead to orchestrate at runtime.

To properly synchronize access without the need to serialize CPU and GPU work, you can use a bump allocator backed by a `MTLBuffer`, or the Metal `setBytes` family of functions.

Bump Allocator

For the best frame encoding performance, implement a bump allocator backed by a `MTLBuffer`.

To achieve this, you first allocate a buffer large enough to contain the data for your frame or render pass. Store an offset tracker alongside this buffer.

When you need to obtain memory, reserve a pointer into the buffer contents at the free offset, and increase the offset’s value by the size of the data to write. When binding buffers to Metal shader converter pipelines, ensure addresses align to 8 bytes. Repeat the process for each piece of data.

To bind the data to your pipeline, use `setBuffer:offset:atIndex:` family of functions, with the appropriate offset. Repeat the process for each buffer to write.

Using this technique, and keeping one buffer for each frame in flight, you can completely avoid race conditions without the cost of synchronization primitives.

The example snippets section of this document presents a simple bump allocator implementation in C++.

SetBytes functions

You may alternatively leverage the `setBytes:` family of function in Metal's command encoders (such as `setVertexBytes:offset:index`, and `setFragmentBytes:offset:index`) to provide your Argument Buffer as inline data in the `MTLCommandBuffer`.

When using the `setBytes:` family of functions, Metal immediately performs a `memcpy` of your Argument Buffer in the CPU timeline, preserving its contents.

The cost of the `memcpy` operation is linear on the size of your top-level Argument Buffer (smaller buffers are faster to copy), and Metal limits the size of each draw call data to 4KB (per draw call).

In some implementations, Metal may have to allocate a buffer to back the `memcpy` operation, which can add significant CPU overhead to your frame encoding time. For CPU-intensive scenarios, favor using a bump allocator instead.

Follow these steps when moving from a slot-based binding model to the top-level Argument Buffer model via `setBytes`:

1. Write your resource GPU addresses and handles into a CPU struct matching the Argument Buffer's layout.
2. Use the `setBytes:` family of functions to have Metal snapshot your CPU struct into an inline "buffer" at slot `kIRArgumentBufferBindPoint` (2).
3. Issue the appropriate `useResource` and `useHeap` calls to signal resource residency (and dependencies) to Metal.

OTHER IMPORTANT CONSIDERATIONS

Indirect Resources

As with any other Argument Buffers, you need to inform Metal of all resources referenced through the top-level Argument Buffer via the `useResource:usage:` and `useResource:usage:stages:` methods for compute and render pipelines respectively. For read-only MTLHeap-backed resources, you may alternatively use `useHeap:`. Resource tables may in turn reference other resources. You need to also call the `useResource:` or `useHeap:` methods to make them resident.

Texture arrays

HLSL shaders may legally treat textures as texture arrays and vice-versa and prior to Metal shader converter 3.0, it was required that you allocate all metal texture resources as a texture array - `MTLTextureType2DArray`. With Metal shader converter 3.0, this is no longer required, and more importantly, it is no longer the default behavior.

The following table shows the appropriate Metal texture type you need to use for each HLSL texture type when converting shaders with Metal shader converter 3.0 or older.

	HLSL	Metal (shader converter 3.0)	Metal (shader converter 2.x)	Mechanism
1D and 2D textures	1D Texture	2D Texture	2D Texture Array	Allocation or texture view
	1D Texture Array	2D Texture Array		
	2D Texture	2D Texture		
	2D Texture Array	2D Texture Array		
Texture cubes	Cube	Cube	Cube Array	
	Cube Array	Cube Array		
Multisampled textures	1D Multisampled Texture	2D Multisampled Texture	2D Multisampled Texture Array	
	1D Multisampled Texture Array	2D Multisampled Texture Array		
	2D Multisampled Texture	2D Multisampled Texture		
	2D Multisampled Texture Array	2D Multisampled Texture Array		
3D textures	No change			

With this change in behavior, you have two options to ensure backward compatibility with your existing Metal shader converter apps:

1. Set the new compatibility flag, `IRCompatibilityFlagForceTextureArray`, on `IRCompiler` instances. If using the `metal-shaderconverter` command line tool, add the command option .
2. At app compile-time, conditionally set the appropriate texture type depending on the Metal shader converter version - using the macro `IR_VERSION_MAJOR`

```
#if IR_VERSION_MAJOR < 3
textureDesc.textureType = MTLTextureType2DArray;
#else
textureDesc.textureType = MTLTextureType2D;
#endif
```

Sampler state objects

Metal needs to know at resource creation time if your game intends to reference a sampler through an Argument Buffer.

Set the `supportArgumentBuffers` property of the `MTLSamplerDescriptor` to `YES` in order to create sample state objects that you can bind to Metal shader converter pipelines.

Starting in macOS 15 and iOS 18, the Metal debug layer asserts when you obtain the GPU address of a sampler state object that doesn't support argument buffers.

VERTEX ATTRIBUTE FETCH

Metal shader converter supports two mechanisms to fetch vertex attributes: *Metal vertex fetch* and a *separate stage-in function*.

Metal vertex fetch

By default, Metal shader converter generates IR that leverages the Metal vertex fetch mechanism. When using this mechanism, you typically provide a `MTLVertexDescriptor` instance to your pipeline state descriptors so Metal can automatically retrieve vertex attributes and make them accessible to your vertex shader.

When using Metal vertex fetch, your stage-in attributes needs to start at index `kIRStageInAttributeStartIndex` (set to

value `11`) defined in the `metal_irconverter_runtime.h` and the buffer bind point for your vertex buffers start at `kIRVertexBufferBindPoint` (set to value `6`).

The following snippet depicts how to configure Metal vertex fetch for a vertex layout consisting of two `float4` attributes:

```
MTLVertexDescriptor* vtxDesc = [[MTLVertexDescriptor alloc] init];

vtxDesc.attributes[kIRStageInAttributeStartIndex + 0].format = MTLVertexFormatFloat4;
vtxDesc.attributes[kIRStageInAttributeStartIndex + 0].offset = 0;
vtxDesc.attributes[kIRStageInAttributeStartIndex + 0].bufferIndex = kIRVertexBufferBir

vtxDesc.attributes[kIRStageInAttributeStartIndex + 1].format = MTLVertexFormatFloat4;
vtxDesc.attributes[kIRStageInAttributeStartIndex + 1].offset = sizeof(simd_float4);
vtxDesc.attributes[kIRStageInAttributeStartIndex + 1].bufferIndex = kIRVertexBufferBir

vtxDesc.layouts[kIRVertexBufferBindPoint].stride = sizeof(simd_float4) + sizeof(simd_f
vtxDesc.layouts[kIRVertexBufferBindPoint].stepRate = 1;
vtxDesc.layouts[kIRVertexBufferBindPoint].stepFunction = MTLVertexStepFunctionPerVerte
```

When you use a render pipeline state following this vertex descriptor, you bind you vertex buffer to it like so:

```
[renderEnc setVertexBuffer:vertices offset:0 atIndex:kIRVertexBufferBindPoint];
```

Visible function vertex fetch

Metal vertex fetch is fast and requires very little setup at both IR conversion time and at pipeline state object creation time. In some situations, however, if your IR requires very flexible datatype conversions, you need to use a separate stage-in function.

To synthesize a separate vertex stage-in function, pass configuration parameter `IRStageInCodeGenerationModeUseSeparateStageInFunction` to function `IRCompilerSetStageInGenerationMode` in the `libmetalirconverter` library before compiling your vertex stage and call `IRMetalLibSynthesizeStageInFunction` afterward to generate the stage-in function.

You can also use the command-line argument `--vertex-stage-in` to direct the Metal shader converter standalone tool to produce both a vertex function and a separate stage-in function. Metal shader converter stores each function in its own metallib.

The Metal shader converter generates separate stage-in functions as *Metal visible functions* that you need to link to your pipeline state via its descriptor's `linkedFunctions` property. At runtime, the converted shader code automatically invokes the visible stage-in function, performing any needed type conversions or apply dynamic offsets via software.

The following example compiles a vertex shader that leverages a separate stage-in function:

```
IRObject* pIR = // input IR

IRCompiler* pCompiler = IRCompilerCreate();

// Synthesize a separate stage-in function by providing a vertex input layout:
IRVersionedInputLayoutDescriptor inputDesc;
inputDesc.version = IRInputLayoutDescriptorVersion_1;
    inputDesc.desc_1_0.numElements = 3;
    inputDesc.desc_1_0.semanticNames[0] = "POSITION";
    inputDesc.desc_1_0.semanticNames[1] = "COLOR";
```

```
    inputDesc.desc_1_0.semanticNames[2] = "TEXCOORD";
    inputDesc.desc_1_0.inputElementDescs[0] = {
        .semanticIndex = 0,
        .format = IRFormatR32G32B32A32Float,
        .inputSlot = 0,
        .alignedByteOffset = 0,
        .inputSlotClass = IRInputClassificationPerVertexData,
        .instanceDataStepRate = 0 /* needs to be 0 for per-vertex data */
    };
    inputDesc.desc_1_0.inputElementDescs[1] = {
        .semanticIndex = 0,
        .format = IRFormatR32G32B32A32Float,
        .inputSlot = 1,
        .alignedByteOffset = sizeof(float)*4,
        .inputSlotClass = IRInputClassificationPerVertexData,
        .instanceDataStepRate = 0 /* needs to be 0 for per-vertex data */
    };
    inputDesc.desc_1_0.inputElementDescs[2] = {
        .semanticIndex = 0,
        .format = IRFormatR32G32B32A32Float,
        .inputSlot = 2,
        .alignedByteOffset = sizeof(float)*4*2,
        .inputSlotClass = IRInputClassificationPerVertexData,
        .instanceDataStepRate = 0 /* needs to be 0 for per-vertex data */
    };

IRError* pError = nullptr;
IRCompilerSetStageInGenerationMode(pCompiler,
                                   IRStageInCodeGenerationModeUseSeparateStageInFunction);
IRObject* pIR = IRCompilerAllocCompileAndLink(pCompiler, nullptr, pIR, &pError);

// Validate pIR != null and no error.

IRShaderReflection* pVertexReflection = IRShaderReflectionCreate();
IRObjectGetReflection(pIR, IRShaderStageVertex, pVertexReflection);

IRMetalLibBinary* pStageInMetalLib = IRMetalLibBinaryCreate();
bool success = IRMetalLibSynthesizeStageInFunction(pCompiler,
                                                   pVertexReflection,
                                                   &inputDesc,
                                                   pStageInMetalLib);

// Verify success

IRMetalLibBinary* pVertexStageMetalLib = IRMetalLibBinaryCreate();
success = IRObjectGetMetalLibBinary(pIR, IRShaderStageVertex, pVertexStageMetalLib);

// Verify success

if (pError)
{
    IRErrorDestroy(pError);
}

IRMetalLibBinaryDestroy(pVertexStageMetalLib);
```

```
IRMetalLibBinaryDestroy(pStageInMetalLib);
IRShaderReflectionDestroy(pVertexReflection);
IRObjectDestroy(pIR);
IRCompilerDestroy(pCompiler);
```

You can also provide the vertex input layout in JSON format to the command-line tool by using parameter: `vertex-input-layout-file`. The following example demonstrates the JSON format that describes the input layout for 5 `float4` input attributes:

```
{
    "InputElements": [
        {
            "AlignedByteOffset": 0,
            "Format": "R32G32B32A32_FLOAT",
            "InputSlot": 0,
            "InputSlotClass": "PerVertexData",
            "InstanceDataStepRate": 0,
            "SemanticIndex": 0
        },
        {
            "AlignedByteOffset": 16,
            "Format": "R32G32B32A32_FLOAT",
            "InputSlot": 1,
            "InputSlotClass": "PerVertexData",
            "InstanceDataStepRate": 0,
            "SemanticIndex": 0
        },
        {
            "AlignedByteOffset": 32,
            "Format": "R32G32B32A32_FLOAT",
            "InputSlot": 2,
            "InputSlotClass": "PerVertexData",
            "InstanceDataStepRate": 0,
            "SemanticIndex": 0
        },
        {
            "AlignedByteOffset": 48,
            "Format": "R32G32B32A32_FLOAT",
            "InputSlot": 3,
            "InputSlotClass": "PerVertexData",
            "InstanceDataStepRate": 0,
            "SemanticIndex": 0
        },
        {
            "AlignedByteOffset": 64,
            "Format": "R32G32B32A32_FLOAT",
            "InputSlot": 4,
            "InputSlotClass": "PerVertexData",
            "InstanceDataStepRate": 0,
            "SemanticIndex": 0
        }
    ],
    "SemanticNames": [
        "POSITION",
```

```
        "NORMAL",
        "TANGENT",
        "TEXCOORD",
        "COLOR"
    ]
}
```

The command-line tool provides option `--generate-vertex-input-template` that you can use to generate a JSON file template to manually customize with your shader's input layout.

At runtime, after you generate a separate stage-in function, you need to create the pipeline state object by linking the functions together. The next example shows how:

```
id<MTLDevice> device = MTLCreateSystemDefaultDevice();

id<MTLLibrary> vertexLib = [device newLibraryWithData: /* vertex stage metallib */ ];

id<MTLLibrary> stageInLib = [device newLibraryWithData: /* stage-in metallib */ ];

MTLRenderPipelineDescriptor* rpd = [[MTLRenderPipelineDescriptor alloc] init];
rpd.vertexFunction =
          [vertexLib newFunctionWithName:vertexLib.functionNames.firstObject];

MTLLinkedFunctions* linkedFunctions = [[MTLLinkedFunctions alloc] init];
linkedFunctions.functions = @[
          [stageInLib newFunctionWithName:stageInLib.functionNames.firstObject]
];
rpd.vertexLinkedFunctions = linkedFunctions;

// ... continue configuring the pipeline state descriptor...

id<MTLRenderPipelineState> pso =
                      [device newRenderPipelineStateWithDescriptor:rpd error:&error];
```

Feature support matrix

Metal shader converter supports a large subset of DXIL IR that enables AAA-grade content on Metal. Use the following reference table, as well as error detection features in Metal shader converter, to ensure the correct conversion of your IR.

Shader Model	Feature	Support	Error-detected	Notes
Pre-6.0	-	Limited	Limited	Some features not supported
SM6.0	Wave intrinsics	Yes	-	
	64-bit integers	Yes	-	
SM6.1	*SV_ViewID*	No	No	
	SV_Baricentrics	Yes	-	Linear interpolation only *GetAttributeAtVertex* not supported
SM6.2	16-bit scalar types	Yes	-	
	denorm mode	No	No	
SM6.3	Ray tracing	Yes	-	
SM6.4	Packed dot-product intrinsics	Yes	-	
	VRS	No	No	
	Library subobjects	No	No	
SM6.5	Ray query	Yes	-	
	Sampler Feedback	No	No	
	Mesh/Ampl shaders	Yes	-	
SM6.6	64-bit atomics	Limited	No	
	Dynamic resources	Yes	No	
	IsHelperLane	Yes	-	
	Pack/unpack intrinsics	No	Yes	
	Compute derivatives	No	Yes	
	Wave size	No	Yes	Wave size needs to be 32
	Raytracing payload qualifiers	No	No	

This list is non-exhaustive. Metal shader converter may not support other features not listed above, including:

- `SV_StencilRef`.
- `minLODClamp` with texture read.

Note: Globally-coherent textures require macOS 15 Sequoia, iOS 18, or later. Some math operations like sine and cosine may offer different precision than other implementations of the input IR.

Offline reflection

Metal shader converter produces valuable information as it compiles your shaders. This information is a complement to, but not a replacement of, the reflection capabilities your source compiler may give you. Refer to the `metal_irconverter.h` header to determine the reflection information Metal shader converter offers.

Metal shader converter produces reflection information in both its forms: standalone executable and library.

- Note: the shader reflection format may change across releases of Metal shader converter. Attempting to deserialize reflection data produced by a different version of Metal shader converter may cause undefined behavior.

STANDALONE REFLECTION

When you use the standalone executable, Metal shader converter writes offline reflection information into a companion JSON to a file path you specify via argument `--output-reflection-file`. Metal shader converter doesn't write reflection information if you don't provide this argument.

Vertex stage reflection

```
{
    "EntryPoint": (string),
    "NeedsFunctionConstants": (bool),
    "Resources": [
        {
            "abIndex": 2,
            "slot": 0,
            "type": (string: "SRV"|"CBV"|"SMP"|"UAV")
        },
        ...
    ],
    "ShaderType": (string: "Vertex"),
    "TopLevelArgumentBuffer": [
        {
            "EltOffset": (int),
            "Size": (int),
            "Slot": (int),
            "Space": (int),
            "Type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "instance_id_index": (int),
    "line_passthrough_shader": {
        "max_primitives_per_mesh_threadgroup": (int)
    },
    "needs_draw_params": (bool),
    "point_passthrough_shader": {
        "max_primitives_per_mesh_threadgroup": (int)
    },
    "triangle_passthrough_shader": {
        "max_primitives_per_mesh_threadgroup": (int)
    },
    "vertex_id_index": (int),
    "vertex_inputs": [
```

```
    ],
    "vertex_output_size_in_bytes": (int),
    "vertex_outputs": [
        {
            "columnCount": (int: 1|2|3|4),
            "elementType": (string),
            "index": (int),
            "name": (string: e.g."sv_position0")
        },
        ...
    ]
}
```

Example: the following vertex shader:

```
struct VertexData
{
    float4 position : POSITION;
    float4 color : COLOR;
    float4 uv : TEXCOORD;
};

struct v2f
{
    float4 position : SV_Position;
    float4 color : USER0;
    float4 uv : TEXCOORD0;
};

v2f MainVS( VertexData vin )
{
    v2f o = (v2f)0;
    o.position = vin.position;
    o.color = vin.color;
    o.uv = vin.uv;
    return o;
}
```

Produces reflection JSON:

```
{
    "EntryPoint": "MainVS",
    "ShaderType": "Vertex",
    "instance_id_index": -1,
    "is_tessellation_vertex_shader": -1,
    "needs_draw_params": false,
    "vertex_id_index": -1,
    "vertex_inputs": [
        {
            "index": 0,
            "name": "position0"
        },
        {
```

```
            "index": 1,
            "name": "color0"
        },
        {
            "index": 2,
            "name": "texcoord0"
        }
    ],
    "vertex_output_size_in_bytes": 48
}
```

Fragment stage reflection

```
{
    "EntryPoint": (string),
    "NeedsFunctionConstants": (bool),
    "Resources": [
        {
            "abIndex": (int),
            "slot": (int),
            "type": (string: "SRV"|"CBV"|"SMP"|"UAV")
        },
        ...
    ],
    "ShaderType": (string: "Fragment"),
    "TopLevelArgumentBuffer": [
        {
            "EltOffset": (int),
            "Size": (int),
            "Slot": (int),
            "Space": (int),
            "Type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "discards": (bool),
    "num_render_targets": (int),
    "rt_index_int": (int)
}
```

Compute stage reflection

```
{
    "EntryPoint": (string),
    "NeedsFunctionConstants": (bool),
    "Resources": [
        {
            "abIndex": (int),
            "slot": (int),
            "type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "ShaderType": (string: "Compute"),
```

```
    "TopLevelArgumentBuffer": [
        {
            "EltOffset": (int),
            "Size": (int),
            "Slot": (int),
            "Space": (int),
            "Type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "tg_size": [
        (int),
        (int),
        (int)
    ]
}
```

Amplification stage reflection

```
{
    "EntryPoint": (string),
    "FunctionConstants": [
        ...
    ],
    "NeedsFunctionConstants": (bool),
    "Resources": [
        {
            "abIndex": (int),
            "slot": (int),
            "type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "ShaderID": (string),
    "ShaderType": (string: "Amplification"),
    "TopLevelArgumentBuffer": [
        {
            "EltOffset": (int),
            "Size": (int),
            "Slot": (int),
            "Space": (int),
            "Type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "max_payload_size_in_bytes": (int),
    "num_threads": [
        (int),
        (int),
        (int)
    ]
}
```

Mesh stage reflection

```
{
    "EntryPoint": (string),
    "FunctionConstants": [

    ],
    "NeedsFunctionConstants": (bool),
    "Resources": [
        {
            "abIndex": (int),
            "slot": (int),
            "type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "ShaderID": (int),
    "ShaderType": (string: "Mesh"),
    "TopLevelArgumentBuffer": [
        {
            "EltOffset": (int),
            "Size": (int),
            "Slot": (int),
            "Space": (int),
            "Type": (string: "SRV"|"CBV"|"UAV")
        },
        ...
    ],
    "max_payload_size_in_bytes": (int),
    "max_primitive_output_count": (int),
    "max_vertex_output_count": (int),
    "num_threads": [
        (int),
        (int),
        (int)
    ],
    "primitive_topology": (string: "Triangle"|"Line"|"Point")
}
```

LIBRARY REFLECTION

When using the Metal shader converter library, you access reflection information for any shader you compiled using function `IRObjectGetReflection`. The reflection object contains information for the shader stage you request.

While the `IRReflection` object holds general reflection data – such as the entry point's name – you access detailed information about a shader stage through reflection information structs. To ensure forward compatibility, all reflection structs are versioned.

Example: reflect the entry point's name of a compiled vertex shader:

```
// Reflection the entry point's name:
IRShaderReflection* pReflection = IRShaderReflectionCreate();
IRObjectGetReflection( pOutIR, IRShaderStageVertex, pReflection );
const char* str = IRShaderReflectionGetEntryPointFunctionName( pReflection );

// ... use store entry point name or use it to find the MTLFunction ... //
```

```
IRShaderReflectionDestroy( pReflection );
```

Example: reflect the thread group size of a compiled compute shader through the compute information struct:

```
// Get reflection data:
IRShaderReflection* pReflection = IRShaderReflectionCreate();
IRObjectGetReflection( pOutIR, IRShaderStageCompute, pReflection );

IRVersionedCSInfo csinfo;
if ( IRShaderReflectionCopyComputeInfo( pReflection, IRReflectionVersion_1_0, &csinfo
{
    // Threadgroup sizes available in csinfo.info_1_0.tg_size
}

// Clean up
IRShaderReflectionReleaseComputeInfo( &csinfo );
IRShaderReflectionDestroy( pReflection );
```

IR runtime model compatibility

DRAW PARAMETERS

Metal shader converter generates code that bridges the gap between the input IR and Metal's runtime model. For example, this allows preserving the IR semantics of `SV_VertexID` and `SV_InstanceID` when taking DXIL IR as the input format.

Continuing with this example, the source IR ensures `SV_VertexID` to start from `0` regardless of whether `StartVertexLocation` is non-zero. In Metal, this isn't the case. The `[[vertex_id]]` attribute includes the base vertex value, if one is specified [MSL Spec §5.2.3.1 Vertex Function Input Attributes], and the base instance value.

Metal shader converter closes this gap, but requires pipelines to bind a supplemental buffer with details about the draw call. You may provide this buffer manually, or use the helper draw calls provided by the Metal shader converter companion header.

Use the reflection information of the vertex stage to determine if a pipeline state object requires additional draw call information. See the "Example snippets" section below for an example on how to access this reflection data.

It's an error not to provide draw call information to a pipeline that requires it, and may trigger a Metal debug layer error.

When your pipeline requires additional draw call information, the Metal shader converter companion header provides convenience draw functions that automatically create and bind these additional buffers at the right bind location. See section "Metal shader converter companion header" for more details.

If you aren't using the companion header, you need to create this buffer manually. The exact format depends on the draw call. A simple non-indexed, non-instanced primitive draw call needs to provide:

- **start_vertex_location**
- **base_vertex_location**
- **base_instance**
- **index_type** – needed for correctly deriving the `SV_VertexID` in a tessellation vertex shader.
- **instance_count** – needed for correctly deriving the `primitive_id` in a domain shader.
- **vertex_or_index_count_per_instance**

Please refer to the implementation of the companion header to determine the layout Metal shader converter requires for other draw call types. This document also includes an example in section “Example snippets”.

Your program needs to bind this data as a buffer at index `4` (`kIRArgumentBufferDrawArgumentsBindPoint`).

DUAL-SOURCE BLENDING

Metal shader converter supports dual source blending. By default, Metal shader converter doesn’t inject this capability into its generated Metal IR, but exposes controls to allow you to enable it always, or to defer the decision to pipeline state creation time.

To request support for dual source blending, pass `IRDualSourceBlendingConfigurationForceEnabled` to function `IRCompilerSetDualSourceBlendingConfiguration`, or `-dualSourceBlending` via the command-line interface.

You can alternatively defer the decision to perform dual-source blending to runtime. In that case, use options `IRDualSourceBlendingConfigurationDecideAtRuntime` or `decideAtRuntime` when configuring dual-source blending support. In this case, Metal shader converter injects a function constant `dualSourceEnabled` into your fragment shader that you then provide when retrieving the function from the produced Metal library.

SAMPLE MASK

Metal shader converter supports setting the sample mask bitmask to inform which samples get updated in active render targets when converting IR.

By default the value is 0xffffffff which update all the samples. This parameter is reseted after each compilation:

```
IRCompiler* pCompiler = IRCompilerCreate();
// Setting the compiler sample mask bit to 0x2 bitmask
IRCompilerSetSampleMask(pCompiler, 0x2);

IRError* pIRError = nullptr;
// Converting pFirstDXIL blob with '0x2' sample mask bitmask
// After IRCompilerAllocCompileAndLink call, the compiler sample mask bitmask is reset
IRObject* pFirstAIR = IRCompilerAllocCompileAndLink(pCompiler, nullptr, pFirstDXIL, &p

// ... Getting the metal lib binary from pFirstAir

// Setting the compiler sample mask bit to 0x2 bitmask
IRCompilerSetSampleMask(pCompiler, 0x4);

IRError* pIRError = nullptr;
// Converting pSecondDXIL blob with '0x4' sample mask bitmask
// Another IRCompilerAllocCompileAndLink call, the compiler sample mask bitmask is res
IRObject* pAIR = IRCompilerAllocCompileAndLink(pCompiler, nullptr, pSecondDXIL, &pIREr
```

RASTER ORDER VIEW

Metal shader converter supports DXIL raster order view resources when converting IR.

To support this feature ensure that your app binds a nil buffer to `kIRArgumentBufferUniformsBindPoint` before issuing draw calls with shaders that access raster order view resources:

```
uint64_t bufferROG = 0;
// Bind a nil buffer to kIRArgumentBufferUniformsBindPoint index to support raster or
[rce setFragmentBytes:&bufferROG length:sizeof(uint64_t) atIndex:kIRArgumentBufferUni

// Issue draw calls with shaders that access raster order view resources
....
```

Shader debugging, profiling, and validation

CONVERT DEBUG INFORMATION

Metal shader converter supports carrying debug information from the source IR into Metal libraries. This enables you to debug, profile, and enable shader validation for your converted pipelines on Xcode 16, macOS 15, and iOS 18, or later.

In order to carry debug information, it first needs to be present in the source IR. Shader compilers typically require that you provide command-line arguments to produce and embed debug information. For example, the open-source DXC compiler requires that you specify input arguments `-Zi -Qembed_debug` to produce and embed the debug information into the DXIL container.

- Note: If your shaders use the `#line` preprocessor directive, you may need to additionally provide the command-line argument `--ignore-line-directives` to DXC to obtain more accurate shader profiling data.

Carrying debug information requires that you set the minimum OS build version of the target Metal library to be macOS 14 or iOS 17. When using the Metal shader converter command-line tool, use argument `--minimum-os-build-version` to specify the minimum operating system version. Using the Metal shader converter dynamic library, you instead call function `IRCompilerSetMinimumDeploymentTarget`.

When your input IR contains debug information, Metal shader converter automatically carries it into the Metal library it produces. You can override this behavior by passing argument `--ignore-debug-information` to the command-line tool, or by calling function `IRCompilerIgnoreDebugInformation(compiler, true)` when using the Metal shader converter dynamic library.

- Note: when you are ready to ship your game or app, be sure to recompile your original shader source code with optimizations enabled (typically `-O3`).

REDUCE METAL LIBRARY SIZE WHEN IT CONTAINS DEBUG INFORMATION

Carrying debug information may significantly increase the size of your Metal library files. You can mitigate this size growth by storing the debug information into a separate companion symbol file and then removing it from your converted Metal libraries. The Metal GPU debugger in Xcode enables you to load the symbol file and resymbolicate your capture.

The following listing shows how to extract symbol and source code data from a Metal library named “shader.metallib” into a separate file and then strip the Metal library to reduce its size:

```
# Extract debug information into a metallibsym file:
xcrun -sdk macosx metal-dsymutil -flat -remove-source shader.metallib

# Strip the debug information from the original Metal library file:
xcrun -sdk macosx metal-strip -S shader.metallib
```

The `-remove-source` option of the `metal-dsymutil` tool modifies the Metal library file by removing its embedded source code information, while the command’s `-flat` option saves shader source code and other symbol information to a `metallibsym` symbol file.

The `metal-strip` tool removes debug information from the Metal library.

Performance Tips

CODEGEN COMPATIBILITY FLAGS

Compiler compatibility flags allow you to exact code generation to the specific requirements of your shaders. You typically enable compatibility flags to support a broader set of features and behaviors (such as out-of-bounds reads) when your shader needs them to operate correctly. These flags, however, carry a performance cost.

Always use the minimum set of compatibility flags your shader needs to attain the highest runtime performance for IR code you compile. By default, all compatibility flags are disabled.

You control the compatibility flags by calling the `IRCompilerSetCompatibilityFlags` API in the Metal shader converter library. The expected parameter is a 64-bit bitmask of flags to enable. Refer to the library's header file for a list of all flags.

You may also control the compatibility flags from the command line. Consult the `metal-shaderconverter -help` for a list of all flags.

AUTOMATIC LINEAR RESOURCE LAYOUT VS EXPLICIT ROOT SIGNATURES

Root signatures provide maximum flexibility when laying out resources in your shader's top-level Argument Buffer, enabling advanced features such as bindless resources. This flexibility, however, comes at the cost of increased indirection.

Favor using a linear resource binding model for shaders that don't require the flexibility of root signatures. This binding model provides a top-level Argument Buffer layout that references resources through a single indirection, improving resource access times.

MINIMUM OS DEPLOYMENT TARGET AND MINIMUM GPU

IRConverter may be able to produce more optimal output when targeting newer GPU families and operating system versions.

Use functions `IRCompilerSetMinimumGPUFamily()` to specify the minimum GPU target and `IRCompilerSetMinimumDeploymentTarget()` to specify the OS and minimum build version your IR needs to support.

IRConverter vends these functions via the command line switches `--minimum-gpu-family`, and `deployment-os` alongside `--minimum-os-build-version`.

TOP-LEVEL ARGUMENT BUFFERS AND GPU OCCUPANCY

Shader code produced by Metal shader converter relies on Argument Buffers to bind resources to pipeline states. Using Argument Buffers to access resources may result in higher register pressure, reducing theoretical shader occupancy when compared to directly binding resources to pipeline slots.

TOP-LEVEL ARGUMENT BUFFERS AND SHADER EXECUTION OVERLAP

The root signature binding model allows you to specify and reference resource (descriptor) tables to Metal and reference them from multiple top-level Argument Buffers without rebinding linked resources multiple times. This may lead to lower CPU times due to reducing the calls into the Metal command encoder.

However, be mindful of potential data dependencies introduced between passes by referencing common resources, which may reduce GPU work overlap and increase the wall clock execution time of your workload.

Consider a compute shader that writes into a texture that a fragment shader subsequently samples. You place this texture in a texture table and reference it from a top-level Argument Buffer available to both the compute dispatch and the draw call.

If the texture is a tracked resource, and the vertex stage is able to access this texture through its top-level Argument Buffer, Metal needs to serialize the GPU execution of the compute dispatch and the vertex stage, even when no race condition exists, and these two stages can theoretically overlap.

When you use the root signature binding model and share resources via top-level Argument Buffers, use the Metal System Trace in Instruments to evaluate the overlap. Instruments gives you insights you can use to fine-tune your workload dependencies and maximize shader execution overlap.

INPUT IR QUALITY INFLUENCES OUTPUT IR PERFORMANCE

Metal shader converter transforms IR directly based on its input. Suboptimal input IR influences the output of Metal shader converter, and may reduce runtime performance of shader pipelines. Always use the best possible input IR as input to Metal shader converter.

For best results, avoid intermediate tools that transform the input IR from other formats and provide Metal shader converter with IR as close to the source language as possible.

ROOT SIGNATURE VALIDATION FLAGS

This flag doesn't affect Metal IR runtime performance. When you instruct Metal shader converter to generate a hierarchical resource layout via root signatures, by default Metal shader converter performs validation checks on your root signature descriptor and produces an error message when it detects issues.

After you've verified your root signatures are correct, you can disable all validation flags to prevent Metal shader converter from performing these checks at compilation time.

Creating hybrid pipelines

Metal shader converter joins the Metal compiler as another mechanism to produce Metal libraries from your existing shader IR.

Since all shaders become Metal IR, you can combine Metal libraries coming from Metal shader converter and from the Metal compiler in a single app and even in a single pipeline. This opens the possibility of using the Metal shading language to access unique features – like programmable blending and tile shaders – not typically available in third-party IR.

To take advantage of programmable blending, after your converted pipeline has output its color, use pipelines with a Metal Shading Language fragment stage to perform frame buffer fetch. A good use case for this is implementing an on-tile post-processing stack.

To read back the on-tile color, observe the following mapping. Color data your converted fragment shader stores in `SV_Target0` is available through attribute `color(0)`.

```
SV_Target0 → [[color(0)]]
```

You can also combine shader stages. This allows you, for example, to take advantage of render pipelines with tessellation and geometry stages while calculating the final coloring in Metal Shading Language.

To accomplish this, match the shader interface using the `user` attribute in your struct member declarations.

```
SV_Position  →  [[user(SV_Position)]]
SV_NORMAL    →  [[user(SV_Normal0)]]
SV_TEXCOORD0 →  [[user(SV_Texcoord0)]]
```

Metal shader converter companion header

The Metal shader converter companion header provides convenience functions to accomplish common tasks:

1. Helps encoding resources into descriptor tables (3 `uint64_t` encoding)
2. Offers wrappers to drawing functions that automatically supply draw parameters to the pipeline
3. Aids the emulation of Geometry and Tessellation pipelines via Metal mesh shaders

To use the companion header, include file `metal_irconverter_runtime.h`.

This header depends on Metal, and you need to include it after including `Metal/Metal.h` or `Metal/Metal.hpp`.

Because this is a header-only library, it requires you to generate its implementation once. You generate the implementation by defining `IR_PRIVATE_IMPLEMENTATION` in a single `m`, `.mm`, or `.cpp` file before including the header. **You need to define this macro exactly once**.

The Metal shader converter companion header is compatible with metal-cpp. To configure the header for metal-cpp usage, define `IR_RUNTIME_METALCPP` before including it. Your program needs to define this macro for every inclusion directive, ensuring types match across the entire program.

You can download metal-cpp from developer.apple.com/metal/cpp.

Example: include the Metal shader converter companion header in a single `cpp` file that uses metal-cpp for rendering, and generate its implementation.

```
#include <Metal/Metal.hpp>
#define IR_RUNTIME_METALCPP       // enable metal-cpp compatibility mode
#define IR_PRIVATE_IMPLEMENTATION // define only once in an implementation file
#include <metal_irconverter_runtime/metal_irconverter_runtime.h>
```

The following sections provide examples of specific tasks you can accomplish with the Metal shader converter companion header.

ENCODING ARGUMENT BUFFERS AND DESCRIPTOR TABLES GENERATED WITH AN AUTOMATIC LAYOUT

Encode a descriptor table with two entries: first a texture array, followed by a sampler state object.

```
const int kNumEntries = 2;
size_t size = sizeof(IRDescriptorTableEntry) * kNumEntries;
MTL::Buffer* pDescriptorTable =
    _pDevice->newBuffer(size, MTL::ResourceStorageModeShared );

auto* pResourceTable = (IRDescriptorTableEntry *)pDescriptorTable->contents();
IRDescriptorTableSetTexture( &pResourceTable[0], pTexture, 0, 0 );
IRDescriptorTableSetSampler( &pResourceTable[1], pSampler, 0 );
```

PROVIDING DRAW PARAMETERS

If a pipeline requires additional draw parameters, for example, when its vertex shader uses *VertexID* or *InstanceID* semantics, you need to provide these to Metal via a buffer.

The Metal shader converter companion header automatically creates and provides these buffers to Metal when using its draw functions.

```
IRRuntimeDrawPrimitives( pEnc, MTL::PrimitiveTypeTriangle, 0, 3 );
```

To have more control over buffer allocation, you may alternatively choose to encode these buffers manually. See the “Example snippets” section below for an example.

Adopting Metal language features in HLSL

Metal shader converter 3.0 provides support to adopt the Metal language features:

- Function constants, for efficient specialization of your shaders
- Frame buffer fetch to enable techniques such as programmable blending

For more details about these features, see the Metal Shading Language Specification (on *Program Scope Function Constants* and *Fragment Function Input Attributes*).

Both features are exposed to HLSL with the header file “Metal_HLSL.inc” which, after installation, can be found at `<install_path>/include/metal_irconverter_ext/Metal_HLSL.inc`, for both macOS and Windows.

Enabling both features requires the definition of unique register spaces for each before including “Metal_HLSL.inc” in your shader.

- define `MTL_FUNCTION_CONSTANT_SPACE=<value>` for function constants
- define `MTL_FRAMEBUFFER_FETCH_SPACE=<value>` for framebuffer fetch

```
// 1. Define unique register spaces to enable the features
// You can also set these using dxc's -D <macro>=<value> command option
#define MTL_FRAMEBUFFER_FETCH_SPACE 2147420893
#define MTL_FUNCTION_CONSTANT_SPACE 2147420894

// 2. Then include the header into your HLSL shader
#include "Metal_HLSL.inc"
```

Note: If either `MTL_FRAMEBUFFER_FETCH_SPACE` or `MTL_FUNCTION_CONSTANT_SPACE` is defined, and if your shader uses the root signature attribute, then you must explicitly include these defined register spaces in your shader’s root signature as needed. (This is because these features depend on the HLSL binding model - CBVs for function constants and SRVs for framebuffer fetch. However, these resources are special and even when included in a root signature definition, will be skipped and take up no space in the root signature). Here’s an example, using the values defined above.

```
#define RootSig \
    "DescriptorTable(CBV(b0), SRV(t0))," \
    "DescriptorTable(Sampler(s0))," \
    "DescriptorTable(CBV(b0, space=2147420894), SRV(t0, space=2147420893))," \
```

USING FUNCTION CONSTANTS

Use the macro `MTL_FUNCTION_CONSTANT(var_type, var_name, fc_index)` to declare a variable as a function

constant in HLSL. Here is an example of a `float4` variable named `ConstantColor` with a function constant index `0.`

```
MTL_FUNCTION_CONSTANT(float4, ConstantColor, 0);
```

During conversion of your HLSL shader to Metal IR, you must also specify the `MTL_FUNCTION_CONSTANT_SPACE` value. Do this with the command option if using the `metal-shaderconverter` command line tool, or with the function `IRCompilerSetFunctionConstantResourceSpace` if using the dynamic library.

At runtime, bind a value to your shader-declared function constant in host code using `MTLFunctionConstantValues` to specialize your HLSL-converted Metal function. The specified data type in host code must always be of type `MTLDataTypeUInt4` .

```
MTLFunctionConstantValues *functionConstantValues = [[MTLFunctionConstantValues alloc]
simd_float4 color = simd_make_float4( 1.0, 1.0, 0.0, 1.0)
[values setConstantValue:&color type:MTLDataTypeUInt4 atIndex:0];
```

The companion header also provides a convenience function for setting function constant values.

```
IRRuntimeFunctionConstantValue colorFC = { .f0 = color.x, .f1 = color.y, .f2 = color.z
IRRuntimeSetFunctionConstantValue(functionConstantValues, 0, &colorFC);
```

USING FRAMEBUFFER FETCH

Use the macro `MTL_LOAD_FRAMEBUFFER(attachment_index, data_type)` to read back color values from the attachment index. The attachment index must be know at compile time and the supported color data types are `float4`, `half4`, `int4`, `uint4`. Here is example:

```
half4 destColor = MTL_LOAD_FRAMEBUFFER(0, half4);
```

During conversion of your HLSL shader to Metal IR, you must also specify the `MTL_FRAMEBUFFER_FETCH_SPACE` value. Do this with the command option `--framebuffer-fetch-register-space` if using the `metal-shaderconverter` command line tool, or with the function `IRCompilerSetFramebufferFetchResourceSpace` if using the dynamic library.

Porting complex pipelines

EMULATING GEOMETRY AND TESSELLATION PIPELINES

Beyond helping bind data to pipelines, the runtime companion header helps you emulate render pipelines that contain traditional geometry and tessellation stages. Metal shader converter allows you to bring these pipelines to Metal, by mapping them to Metal mesh shaders.

To help with the process of building mesh render pipeline state objects from the geometry and tessellation shader stages, the companion header offers the following functions:

- `IRRuntimeNewGeometryEmulationPipeline`
- `IRRuntimeNewGeometryTessellationEmulationPipeline`

These helper functions take as input parameters descriptor structures with the building blocks to compile the pipeline.

The descriptor structure members reference the Metal libraries containing the pipeline's shader functions, reflection data, and

a base mesh render pipeline descriptor that describes the render attachments.

Structure `IRGeometryEmulationPipelineDescriptor` contains:

- **stageInLibrary**: a `MTLLibrary` containing the stage in function.
- **vertexLibrary**: a `MTLLibrary` containing the vertex function.
- **vertexFunctionName**: the name of the vertex function to retrieve from the vertex library.
- **geometryLibrary**: a `MTLLibrary` containing the geometry function.
- **geometryFunctionName**: the name of the geometry function to retrieve from the geometry library.
- **fragmentLibrary**: a `MTLLibrary` containing the fragment function.
- **fragmentFunctionName**: the name of the fragment function to retrieve from the fragment library.
- **basePipelineDescriptor**: a `MTLMeshRenderPipeline` descriptor providing template configuration for the pipeline, such as render attachments.
- **pipelineConfig**: reflection data that you obtained during the Metal shader converter compilation process.

Structure `IRGeometryTessellationEmulationPipelineDescriptor` shares all members of the `IRGeometryEmulationPipelineDescriptor` structure, and expands it to also include the following members:

- **hullLibrary**: a `MTLLibrary` containing the hull function and tessellator.
- **hullFunctionName**: the name of the hull function to retrieve from the hull library.
- **domainLibrary**: a `MTLLibrary` containing the domain function.
- **domainFunctionName**: the name of the hull function to retrieve from the domain library.

Providing a geometry shader is optional when you emulate a tessellation pipeline. Set the `geometryLibrary` member to `NULL` and set the `geometryFunctionName` to `kIRTrianglePassthroughGeometryShader`, (`"irconverter_domain_shader_triangle_passthrough"`), `kIRLinePassthroughGeometryShader`, (`"irconverter_domain_shader_line_passthrough"`), or `kIRPointPassthroughGeometryShader` (`"irconverter_domain_shader_point_passthrough"`) for triangle, line, and point pipeline states respectively.

- Note: when you use a passthrough geometry shader, you can obtain the values to assign to member `gsMaxInputPrimitivesPerMeshThreadgroup` from the domain shader's reflection.

The companion header provides the following draw helper functions to help you use the emulation render pipeline states. Issue these function calls as part of your render pass encoding process to encode a mesh dispatch workload that emulates your geometry and tessellation pipelines.

- `IRRuntimeDrawIndexedPrimitivesGeometryEmulation`
- `IRRuntimeDrawIndexedPatchesTessellationEmulation`

Before issuing these calls, allocate `IRRuntimeVertexBuffers` to define and bind your vertex and patch buffers and strides at the `kIRVertexBufferBindPoint` (6) index of the object stage. In addition, ensure that you make your vertex buffers resident via `useResource` or `useHeap`.

For a complete example of how to perform geometry and tessellation pipeline emulation, please check out the "Complete examples" section of this document.

USING AMPLIFICATION AND MESH SHADERS

Metal shader converter supports DX mesh shaders by converting them to Metal mesh shaders. If you use amplification shaders, Metal shader converter maps these to the object shader stage.

Just like with other shader stages, Metal shader converter produces useful offline reflection data you can use to obtain information about payload sizes, resource layout, number of threads per thread group and more.

CREATING APPEND AND CONSUME BUFFERS

Append/consume buffers provide storage for shaders and an atomic counter to perform unordered insert and remove operations on this storage.

The Metal shader converter runtime provides a convenient function, `IRRuntimeCreateAppendBufferView`, that takes an input buffer for storage, creates the atomic counter, and bundles them together in a `IRBufferView` object. The `IRBufferView` object provides an abstraction that enables you to use the input Metal buffer as an append/consume buffer.

Use function `IRDescriptorTableSetBufferView` to bind an `IRBufferView` object as an append/consume buffer to a descriptor table.

Function `IRRuntimeGetAppendBufferCount` allows you to query the current value of the atomic counter associated with the append/consume buffer.

- Note: Metal shader converter implements the atomic counter through texture atomics, which require macOS 14 Sonoma, iOS 17, or later.

USING UNBOUNDED ARRAYS

Metal shader converter supports unbounded arrays, providing a path to implement bindless pipelines. Unbounded arrays are typically declared in shading languages by omitting the size specifier, such as `StructuredBuffer<T> inBuffers[] : register(t0, space0)`.

After declaring an unbounded array, you specify its number of elements at shader conversion time by providing a root signature, opting into Metal shader converter's explicit resource layout mode. This mechanism provides the CPU with the flexibility to define the number of resources in the array via a descriptor table. It is not possible to use unbounded resources with the automatic resource layout mode.

When binding resources to your explicit-layout signature pipelines, your top level Argument Buffer contains the GPU addresses of the descriptor tables your root signature specifies. These, in turn, reference shader resources. In Metal, each descriptor table corresponds to a `MTLBuffer`.

Visually, your hierarchy corresponds to the following diagram:

- **Top-Level Argument Buffer** `-(uint64_t)->` **descriptor table** `-(IRDescriptorTableEntry)->` **resource**

Your descriptor tables may have as many entries as your root signature specifies at conversion time. Each entry in the table corresponds to the memory layout of `IRDescriptorTableEntry`.

USING "DYNAMIC RESOURCES"

Metal shader converter enables newer binding models where you bind resource and sampler heaps directly to your shaders and directly index into them. Metal shader converter automatically compiles in support for resource and sampler heaps at shader conversion time.

To bind your resources through this mechanism, bind a descriptor table at index `kIRDescriptorHeapBindPoint` (`0`) for general resources, and `kIRSamplerHeapBindPoint` (`1`) for samplers.

The pipeline expects each descriptor table to consist of an argument buffer where each entry corresponds to the layout defined by `IRDescriptorTableEntry`.

Your application needs to call `useResource` or `useHeap` for all indirectly-accessed resources to ensure they are resident when the pipeline executes.

LEVERAGING INLINE RAY TRACING

Metal shader converter supports compiling inline ray tracing shaders to Metal IR. At compilation time, Metal shader converter directly maps inputs of type `RaytracingAccelerationStructure` to Metal instance acceleration structures.

You bind a Metal instance acceleration structure to the inline ray tracing pipeline's top-level Argument Buffer indirectly through an acceleration structure header. The Metal shader converter runtime companion header provides structure definition `IRRuntimeAccelerationStructureGPUHeader` you can use.

To perform inline raytracing, you provide your pipelines an instance of this structure through a `MTLBuffer` by binding its GPU address to your top-level argument buffer, or descriptor tables.

The `IRRuntimeAccelerationStructureGPUHeader` in this buffer contains the GPU resource ID of the acceleration structure against which to trace rays, as well as an array of instance contributions to the hit index. You can store the instance contribution array in its own buffer, or, alternatively, allocate extra space in the header buffer and use it for storing the instance contributions array.

Use convenience function `IRRaytracingSetAccelerationStructure` to build the acceleration structure header and encode instance contributions to hit groups, and function `IRDescriptorTableSetAccelerationStructure` to encode the acceleration structure header into a MTLBuffer.

Once your application creates the acceleration structure header buffer, bind it to the top-level Argument Buffer like any other buffer. It isn't possible to bind *primitive* acceleration structures to an inline ray-tracing shader compiled by Metal shader converter as this type is not present in the input IR.

Ensure you call `useResource` or use `useHeap` to make the Metal acceleration structure and the acceleration structure binding header resident. If your program stores the instance contribution data array in a `MTLBuffer` separate from the header, make sure to make it resident as well.

- **Note**: to ensure your rays intersect the acceleration structure, verify that the RayQuery object's template parameters those of the acceleration structure. For example, `RAY_FLAG_CULL_NON_OPAQUE` culls all acceleration structures that don't have the `MTLAccelerationStructureInstanceOptionOpaque` option.

CONVERTING AND RUNNING RAY TRACING PIPELINES

Metal shader converter enables you to bring ray tracing pipelines consisting of dedicated ray-tracing shader stages to Metal. Metal shader converter maps ray generation, intersection, any hit, closest hit, callable, and miss shaders to Metal visible and intersection functions. You use these metal functions, optionally in conjunction with synthesized indirect intersection functions, to build ray tracing compute pipeline state objects that you can then use to trace rays against Metal acceleration structures and appropriately evaluate using a shader binding table (SBT).

The high level process consists of the following steps:

1. Converting HLSL shaders to Metal functions
2. Building ray tracing pipeline states
3. Building shader binding tables
4. Dispatching Rays

Converting HLSL shaders to Metal functions

Metal shader converter offers different compilation modes for conversion of ray generation and intersection shaders when setting up your ray tracing pipeline compilation from HLSL to Metal IR.

Ray generation:

- Kernel
- Visible function

Intersection:

- Visible function
- Intersection function table function
- Intersection function buffer function

With Metal shader converter 3.0, the default recommendation is to compile your ray generation shaders as kernels and intersection shaders as intersection function buffer functions. With the metal-shaderconverter command-line tool, you do this by providing the option `–rt-ray-generation-compilation=kernel` and `–rt-intersection-compilation=ifIntersectionFunctionBufferFunction.` With the Metal shader converter dynamic library, specify `IRRayGenerationCompilationKernel` and `IRIntersectionFunctionCompilationIntersectionFunctionBufferFunction` when configuring your ray tracing pipeline with `IRRayTracingPipelineConfiguration`.

- Note: Metal intersection function buffer is only supported on GPUFamilyApple9 or later devices; and on macOS 16 or iOS 19 or later OS. For other device or OS configurations fallback to compiling to visible functions if targeting an older device or OS. Compiling to intersection function table functions may offer a performance improvement and its details are discussed below under “Performance considerations for ray tracing pipelines”.

If compiling intersection shaders into visible functions, you must synthesize an indirect intersection function for triangles or procedural geometry or both depending on your pipeline. With the metal-shaderconverter command-line tool, you do this with the command options `--synthesize-indirect-intersection-function –rt-hit-group-type=triangles` for triangle geometry and `--synthesize-indirect-intersection-function –rt-hit-group-type=procedural_primitive` for procedural geometry. With the Metal shader converter dynamic library, set the hit group type with `IRCompilerSetHitgroupType` to either `IRHitGroupTypeTriangles` or `IRHitGroupTypeProceduralPrimitive` before synthesizing the function with `IRMetalLibSynthesizeIndirectIntersectionFunction`.

When you use a custom intersection function with an any hit shader, Metal shader converter requires that you fuse these two shaders together into a single one. Use function `IRCompilerAllocCombineCompileAndLink`, or command-line argument `-fuse-any-hit-name`, to combine the functions together at compile time.

After conversion of your ray tracing stages to Metal IR, generate a metal function for each. These metal functions will then be used to build the ray tracing pipeline state.

Building ray tracing pipeline states

Depending on your ray tracing pipeline configuration, Metal shader converter converts all your defined ray tracing stages into Metal libraries consisting of a compute kernel (for ray generation) and with visible and intersection functions (for hit group, miss, and callable shaders).

Build your ray tracing pipeline states by creating a compute pipeline state with the compute kernel function together with all visible and intersection functions as Metal linked functions.

With this ray tracing pipeline state, you create a Metal visible function table for the visible functions and a Metal intersection function table for the intersection functions. These function tables contain function pointers to your converted functions and

also represent the shader identifiers we need to encode in your shader binding table (SBT).

- Note: For synthesized indirect intersection functions, when Metal performs a ray tracing traversal, it adds together the intersection function offset properties of the geometry and instance acceleration structures , determining the intersection function index it calls from the `MTLIntersectionFunctionTable`. Typically, this index does not match the shader record location in the shader binding table. To bridge this gap between APIs, store the indirection function handles at the positions in the intersection function table corresponding to the summed indices of the geometry and instance acceleration structures. The indirect intersection function re-evaluates the intersection offset into the shader binding table using the standard equation the source IR expects, finds the appropriate shader record, and uses its identifier to index into the visible function table to call your converted closest-hit, any-hit, intersection, and miss shaders.

Building Shader Binding Tables

Metal represents Shader Binding Tables as instances of `MTLBuffer`. The entries of these buffers consist of shader records, with each record containing a Shader Identifier (`IRShaderIdentifier`) and local root signature data. This arrangement allows you to represent the Shader Binding Table in the same fashion that the source IR expects.

The Shader Identifier in the shader binding table makes the association between the index shader record for the intersection and the offset in the visible function table Metal calls. It consists of the following members:

1. `shaderHandle`: for ray generation, miss, callable shaders, index into visible function table containing the translated function. For HitGroups, index to the converted closest-hit shader.
2. `intersectionShaderHandle`: for HitGroups, index into the visible function table containing a converted custom intersection function.
3. `localRootSignatureSamplersBuffer`: GPU address to a buffer containing static samplers for shader records.

The Metal shader converter runtime companion header offers functions `IRShaderIdentifierInit` and `IRShaderIdentifierInitWithCustomIntersection` to help you initialize ShaderIdentifier instances as necessary to build your Shader Binding Tables.

- Note: shader handle 0 is reserved to denote “invalid handle” to the runtime. To ensure your shader handles are non-zero, do not use index 0, when set function handles into your visible and intersection function tables. Conversely, use `0` as the `shaderHandle` value to denote a “null” shader handle in your SBT.

Dispatching rays

In order to dispatch rays, dispatch a compute kernel using the ray tracing pipeline state object. You provide the ray dispatch parameters to the kernel by binding a `MTLBuffer` containing an instance of the `IRDispatchRaysArgument` structure to bind point `kIRRayDispatchArgumentsBindPoint`.

The Metal shader converter runtime companion header defines the `IRDispatchRaysArgument` and `IRDispatchRaysDescriptor` structures. The first provides the references to the GPU virtual address of the visible and intersection function tables containing your converted shaders and intersection functions, as well as the global root signature data, amongst other members.

The `DispatchRaysDesc` member of the `IRDispatchRaysArgument` structure provides crucial information for accessing the Shader Binding Table, including the start address, and size in bytes for ray generation shaders, as well as the same plus appropriate strides for the hit group, miss, and callable table.

After binding this structure to your pipeline, dispatch compute work as usual to start the ray tracing pipeline.

PERFORMANCE CONSIDERATIONS FOR RAY TRACING PIPELINES

The following sections provide details on performance considerations and optimization opportunities for your ray tracing

pipeline

Optimizing parameter passing

While the different shader stages comprising ray tracing may be independent, in order to link them together into a ray tracing pipeline state and pass data between them, you need to specify the maximum shared attribute size in bytes for all shaders in the same pipeline.

Use function `IRRayTracingPipelineConfigurationSetMaxAttributeSizeInBytes` to accomplish this when using the Metal shader converter dynamic library, or command-line argument `-rt-maximum-attribute-size-in-bytes` for the standalone executable.

Additionally, this function allows you to narrow the number of ray tracing intrinsic functions at compile time, yielding significant runtime performance improvements to your ray tracing pipelines.

In order to achieve this, before compilation call function `IRObjectGatherRaytracingIntrinsics` on your source IR. This function takes input IR and produces an intrinsic use mask for the shader. Apply this to all your ray tracing shaders to calculate the intrinsic use mask for all applicable stages. If you have multiple shaders of one kind, bitwise-OR the masks together.

With the intrinsic use masks for all your shaders, call function `IRRayTracingPipelineConfigurationSetIntrinsicMasks` on when configuring a `IRRayTracingPipelineConfiguration` instance before compiling the shaders that comprise the ray tracing pipeline. This process directs the code generation logic to tailor the IR specifically to the minimum set of intrinsics your pipeline needs, improving runtime performance by reducing parameter passing.

When using the command-line interface of Metal shader converter, pass argument `-gather-raytracing-intrinsics` to make the compiler calculate and intrinsic use mask. Metal shader converter prints the mask to stdout as well as to an output file.

Similarly to using the dynamic library interface, once you have the intrinsic use masks for all the shaders that comprise your ray tracing pipeline, you compile the shaders passing the intrinsic masks as a uint64 number via the command-line interface using flags:

- `-rt-closest-hit-mask`
- `-rt-miss-mask`
- `-rt-callable-mask`
- `-rt-anyhit-mask`

The Metal shader converter command-line interface expects these numbers to start with the prefix `0x`.

Increasing runtime performance by inlining function calls

You may be able to increase ray tracing runtime performance by limiting your ray tracing pipelines to a single ray generation shader. In this scenario, Metal may inline your ray generation shader into the ray dispatch kernel as it builds your pipeline state object, reducing the cost of starting ray tracing work.

If your pipeline state objects link multiple ray generation shaders, you may still take advantage of this optimization by compiling each specific ray generation function as a ray dispatch kernel, allowing you to build multiple specialized pipeline state objects without modifications to your high-level renderer.

See section “Converting and running ray tracing pipelines” above for details on how to compile your ray generation shaders as

ray dispatch kernels.

You may be able to further reduce the runtime costs of shader function calling by defining function groups using the `groups` property of the `MTLLinkedFunctions` instance you use for linking visible functions to your pipelines.

By aggregating functions that your pipeline state object links into groups that correspond to ray generation, closest hit, or miss shaders, Metal only needs to consider a subset of pipeline-linked functions as potential call candidates.

The Metal shader converter runtime header defines the following constants to group visible functions:

- `kIRFunctionGroupRayGeneration`: (“`rayGen`”)
- `kIRFunctionGroupClosestHit`: (“`closestHit`”)
- `kIRFunctionGroupMiss`: (“`miss`”)

Optimizing the runtime of intersection and any-hit shaders without Metal intersection function buffers

Note: This section only applies if, due to device or OS limits, you are unable to compile your intersection shaders to intersection function buffer functions but would like performance improvements over compiling to visible functions.

If your ray tracing pipeline uses custom intersection or any hit shaders, or custom intersection+any hit shader pairs, and all your ray generation shaders use the same “geometry multiplier” (the total number of ray types), Metal shader converter enables you to potentially accelerate runtime performance further by compiling these shader types directly into Metal intersection functions.

Under this configuration, you use Metal intersection function tables — instead of visible functions — to more efficiently describe your game’s different ray types to Metal.

Because your geometry multiplier determines the total number of ray types for your ray tracing workload, when you use multiple intersection function tables to implement different ray types, you allocate as many Metal intersection function tables as your geometry multiplier. You then populate each table with the functions that correspond to the intersection and any hit shaders for that ray type.

For convenience, you organize these Metal intersection function tables into a single “intersection function tables” argument buffer that you provide to Metal as part of your `IRRayDispatchArguments` structure when you dispatch ray tracing work.

To implement this optimization, first determine that your geometry multiplier is fixed across all your ray generation shaders. You then compile all your ray tracing shaders using option `IRIntersectionFunctionCompilationIntersectionFunction` when configuring your pipeline with `IRRayTracingPipelineConfiguration`.

Next, set up your shader binding table as usual and iterate over its hit groups to append them (in order) into a temporary CPU intersection functions array according to the following criteria:

1. Obtain a CPU-visible pointer to the beginning of your hit group table in the shader binding table.
2. For each hit group, retrieve its shader identifier, and determine if its intersection shader handle member contains a non-`NULL` value. If it does, retrieve the Metal intersection function corresponding to it and append it to the temporary Metal intersection function array.
3. If the intersection shader handle in the shader identifier is instead `NULL`, append a `NULL` sentinel value to the intersection function array. This reserves the index for a built-in opaque triangle intersection function.

Note: as you calculate the appropriate offset in bytes for each hit group record, remember to take into consideration the hit group table’s stride.

Next, you set up your intersection function tables argument buffer:

1. Prior to dispatching ray tracing work, allocate a Metal argument buffer to store as many Metal intersection function tables as your geometry multiplier.
2. Allocate as many Metal intersection function tables as your geometry multiplier. Configure each Metal intersection function table to store as many functions as the total number of functions, divided by the geometry multiplier:
   a. `per_table_function_count = function_count / geometry_multiplier`
3. Store the Metal resource ID of each Metal intersection function table in the intersection function table argument buffer. The offset at which you store each table corresponds to each ray type plus the geometry multiplier times `16`.
   a. `index = ray_type + (geometry_multiplier * 16) for ray_type in [0..geometry_multiplier)`
4. Populate each table by iterating over its function count. The function to store into each slot corresponds to the slot index times the geometry multiplier plus the table number (note this is not the table index):
   a. `function_index = ((table_slot * geometry_multiplier) + table_number) for table_slot in [0..per_table_function_count), table_number in [0..geometry_multiplier)`
5. If the temporary CPU intersection function array at this function index contains a Metal intersection function, retrieve its function handle from the pipeline state object and store it.
6. Otherwise, if there is no Metal intersection function at this index (it's the `NULL` sentinel value), store the built-in opaque intersection function by calling Metal function: `setOpaqueTriangleIntersectionFunctionWithSignature`.
7. As you build your Metal instance acceleration structure, you need to divide its `intersectionFunctionTableOffset` by the fixed geometry multiplier.

At ray dispatch time, set the member `IntersectionFunctionTables` of structure `IRDispatchRaysArgument` to point to your Metal argument buffer containing references to the Metal intersection function tables.

- Note: Metal intersection function tables are indirect resources that you need to manually mark resident to Metal by calling the `useResource` API or using Metal residency sets.

Maximizing pipeline state build performance

The time required to build compute pipelines is proportional to the number of visible functions you link into them. You can accelerate the process of building compute pipeline state instances by leveraging Metal binary functions (`MTLLinkedFunctions.binaryFunctions`).

When you compile visible functions into binary functions at runtime, and then reuse them across your pipelines, you avoid the cost of repeatedly lowering each function for each pipeline you build. Using binary functions, however, may prevent Metal from performing some optimizations that could reduce the GPU execution time of your pipeline.

Metal GPU binary generation

You can use the offline compiler tool, `metal-tt`, to ingest the output of Metal shader converter and produce finalized GPU binaries as part of your shader build system. Use this process to fully compile the non-MSL shader source to GPU binaries that can be loaded into Metal with no pipeline compilation overhead on device.

The following **Python** script shows an example of a shader pipeline that fully compiles HLSL to Apple GPU binaries. Inputs are the shader source, entry points, and shader profiles, alongside an *mtlp-json* description of the PSO.

```
import os
import subprocess
cmd = subprocess.call
```

```
DXC="dxc"
MSC="metal-shaderconverter"
METAL_TT="xcrun -sdk macosx metal-tt"

products=["compute_pso", "render_pso"]
dependencies={
    "render_pso" : ("render_pso.mtlp-json",
     [("shaders.hlsl", "MainVS", "vs_6_0"), ("shaders.hlsl", "MainFS", "ps_6_0")]),

    "compute_pso" : ("compute_pso.mtlp-json",
     [("shaders.hlsl", "MainCS", "cs_6_0")])
}

shader_source_dir="shader_source/"
output_dir="output_dir/"
target_archs= \
    "".join(subprocess.check_output(['xcrun', 'metal-arch']).replace("\n"," "))

if not os.path.isdir(output_dir):
    os.mkdir(output_dir)

for product in products:
    pso_json, deps = dependencies[product]
    for dep in deps:
        source_file, entry, profile = dep
        cmd([DXC,
             shader_source_dir+source_file,
             "-T", profile,
             "-E", entry,
             "-Fo", output_dir + entry + ".dxil"])

        cmd([MSC,
            "-rename-entry-point", entry,
            output_dir + entry + ".dxil",
            "-o", "./" + output_dir + entry + ".metallib"])

    cmd(METAL_TT.split(" ") + target_archs.split(" ") +
        ["-L", output_dir, shader_source_dir + pso_json,
         "-o", output_dir + product+".gpubin"])
```

Example of *mtlp-json* file you can use to produce a render pipeline state:

```
{
  "version": {
    "major": 0,
    "minor": 1,
    "sub_minor": 1
  },
  "generator": "MetalFramework",
  "libraries": {
    "paths": [
      {
        "label": "vtxMetalLib",
```

```
        "path": "MainVS.metallib"
      },
      {
        "label": "fragMetalLib",
        "path": "MainFS.metallib"
      }
    ]
  },
  "pipelines": {
    "render_pipelines": [
      {
        "vertex_function": "alias:vtxMetalLib#MainVS",
        "fragment_function": "alias:fragMetalLib#MainFS",
        "vertex_descriptor": {
          "attributes": [
            {
              "format": "Float4"
            }
          ],
          "layouts": [
            {
              "stride": 16
            }
          ]
        },
        "color_attachments": [
          {
            "pixel_format": "BGRA8Unorm_sRGB"
          }
        ]
      }
    ]
  }
}
```

Note: some limits apply to the offline compilation process. Please review the Apple documentation for the latest set of supported features.

Example Snippets

SAVE A METALLIBBINARY TO DISK

You can link the Metal shader converter library to enhance your custom offline shader compilation pipelines, and produce metallib files. The following snippet stores the metallib to disk for later consumption:

```
bool saveMetalLibToFile(const char* filepath,
                        const IRMetalLibBinary* pMetalLib)
{
    FILE* f = fopen(filepath, "w");
    size_t siz = IRMetalLibGetBytecodeSize(pMetalLib);
    uint8_t* bytes = (uint8_t)malloc(siz);
    IRMetalLibGetBytecode(pVertexStageMetalLib, bytes);

    fwrite(bytes, siz, 1, f);
```

```
    if (ferror(f)) {
        // ...error...
    }

    fclose(f);
    free(bytes);
}
```

A custom shader pipeline processor may choose to store the metallib bytecode and size in a custom asset packaging format.

SUPPLY DRAW PARAMETERS TO A PIPELINE STATE WITHOUT THE COMPANION HEADER

This example shows a draw call that uses a shader that leverages `VertexID`. `VertexID` has different semantics across Metal and DXIL, so the app needs to provide the pipeline state object extra information about it, so the compiled shader can adjust the semantic's value to match the value the original IR expects.

This is only needed when using `VertexID` or `InstanceID`. Use the `needs_draw_info` member of the vertex stage reflection information to determine programatically if the original IR requires this additional buffer.

```
struct DrawArgument
{
    uint vertexCountPerInstance;
    uint instanceCount;
    uint startVertexLocation;
    uint startInstanceLocation;
} da = { 3, 1, 0, 0 };

struct DrawParams
{
    DrawArgument draw;
} dp = { .draw = da };

struct DrawInfo
{
    uint32_t indexType;
    uint32_t primitiveTopology;
    uint32_t maxInputPrimitivesPerMeshThreadgroup;
    uint32_t objectThreadgroupVertexStride;
    uint32_t gsInstanceCount;
} di = { 0, 3, 0, 0, 0 };

pEnc->setVertexBytes( &dp, sizeof( DrawParams ), 4);
pEnc->setVertexBytes( &di, sizeof( DrawInfo ), 5 );
pEnc->drawPrimitives( MTL::PrimitiveTypeTriangle, NS::UInteger(0), NS::UInteger(3) );
```

Bind points `4` and `5` are specially designated slots where the converted pipeline expects the application to bind buffers containing information about the draw call.

When you use the runtime companion header, the `IRRuntimeDrawPrimitives` function builds and submits the necessary draw params structure to the draw call based on its input, automatically binding these buffers in the correct slots. The companion header also defines `kIRArgumentBufferDrawArgumentsBindPoint` and `kIRArgumentBufferUniformsBindPoint` to conveniently reference these bind slots.

REFLECT WHETHER A VERTEX SHADER REQUIRES DRAW PARAMETERS

This example demonstrates how to determine whether a shader requires a draw parameter buffer via vertex shader reflection information.

```
// Get reflection data:
IRShaderReflection* pReflection = IRShaderReflectionCreate();
IRObjectGetReflection( pOutIR, IRShaderStageVertex, pReflection );

// Determine whether draw params are needed:
IRVersionedVSInfo vsinfo;
if (IRShaderReflectionGetVertexInfo(pReflection, IRReflectionVersion_1_0, &vsinfo))
{
    if ( vsinfo.info_1_0.needs_draw_params )
    {
        // PSO needs a draw params buffer bound to the vertex stage
    }
}

// Clean up
IRShaderReflectionReleaseVertexInfo( &vsinfo );
IRShaderReflectionDestroy( pReflection );
```

DEFINE A GLOBAL ROOT SIGNATURE USING THE METAL SHADER CONVERTER LIBRARY

The global root signature defines a sampler and a texture 2D. You need to put both samplers and texture references into their own tables. You may only reference raw resources directly from the top-level Argument Buffer, such as constants, constant buffers, buffer SRVs, and UAVs.

Root signatures in Metal shader converter are subject to the same limitations as in Microsoft's DirectX. If you aren't familiar with these requirements, please refer to Microsoft's documentation. Supplying an invalid root signature to Metal shader converter may trigger a validation error. If your program disables validation, the tool's behavior is undefined.

```
IRVersionedRootSignatureDescriptor desc;
desc.version = IRRootSignatureVersion_1_1;
desc.desc_1_1.Flags = IRRootSignatureFlagNone;

// Samplers are placed in their own table:
desc.desc_1_1.NumStaticSamplers = 1;
IRStaticSamplerDescriptor pSSDesc[] = { {
    .Filter = IRFilterMinMagMipLinear,
    .AddressU = IRTextureAddressModeWrap,
    .AddressV = IRTextureAddressModeWrap,
    .AddressW = IRTextureAddressModeWrap,
    .MipLODBias = 0,
    .MaxAnisotropy = 0,
    .ComparisonFunc = IRComparisonFunctionNever,
    .BorderColor = IRStaticBorderColorOpaqueBlack,
    .MinLOD = 0,
    .MaxLOD = std::numeric_limits<float>::max(),
    .ShaderRegister = 0,
    .RegisterSpace = 0,
    .ShaderVisibility = IRShaderVisibilityPixel
} };
desc.desc_1_1.pStaticSamplers = pSSDesc;
```

```
// Parameters (1 texture):
IRDescriptorRange1 ranges[1] = { [0] = {
    .RangeType = IRDescriptorRangeTypeSRV,
    .NumDescriptors = 1,
    .BaseShaderRegister = 0,
    .RegisterSpace = 0,
    .Flags = IRDescriptorRangeFlagDataStatic,
    .OffsetInDescriptorsFromTableStart = 0
}
};
IRRootParameter1 pParams[] = { {
    .ParameterType = IRRootParameterTypeDescriptorTable,
    .DescriptorTable = { .NumDescriptorRanges = 1, .pDescriptorRanges = ranges },
    .ShaderVisibility = IRShaderVisibilityPixel
} };
desc.desc_1_1.NumParameters = 1;
desc.desc_1_1.pParameters = pParams;

IRError* pRootSigError = nullptr;
IRRootSignature* pRootSig = IRRootSignatureCreateFromDescriptor( &desc, &pRootSigError
if ( !pRootSig )
{
    // handle and release error
}

// After compiling DXIL bytecode to Metal IR using this root signature,
// it should have 2 entries:
//
// offset 0: a uint64_t referencing a table that contains a
// void* resource (SRV).
//
// offset 8 (sizeof(uint_64)): a uint64_t referencing a table with
// one sampler.

// The sampler table should be encoded like so:
// For each sampler:
// 64-bits: GPU VA of the sampler.
// 64-bits: 0
// 64-bits: Sampler's LOD Bias.

// The SRV table should be encoded like so:
// 64-bits: 0
// 64-bits: Texture GPU Resource ID
// 64-bits: 0

// Use the Companion Header for help encoding resources into descriptor tables.

IRCompiler* pCompiler = IRCompilerCreate();
IRCompilerSetGlobalRootSignature( pCompiler, pRootSig );

// Compile DXIL to Metal IR
IRError* pError = nullptr;
IRObject* pDXIL = IRObjectCreateFromDXIL(dxilFragmentBytecode,
                                         dxilFragmentSize,
                                         IRBytecodeOwnershipNone);
```

```
IRObject* pOutIR = IRCompilerAllocCompileAndLink(pCompiler,
                                                 NULL,
                                                 pDXIL,
                                                 &pError);

// if pOutIR is null, inspect pError for causes. Release pError afterwards.

IRMetalLibBinary* pMetallib = IRMetalLibBinaryCreate();
IRObjectGetMetalLibBinary( pOutIR, IRShaderStageFragment, pMetallib );

size_t metallibSize = IRMetalLibGetBytecodeSize( pMetallib );
uint8_t* metallib = new uint8_t[ metallibSize ];
if ( IRMetalLibGetBytecode( pMetallib, metallib ) == metallibSize )
{
    // Store metallib for later use or directly create a MTLLibrary
}

delete [] metallib;

IRMetalLibBinaryDestroy( pMetallib );

IRObjectDestroy( pOutIR );
IRObjectDestroy( pDXIL );

IRRootSignatureDestroy( pRootSig );
IRCompilerDestroy( pCompiler );
```

DEFINE A ROOT SIGNATURE VIA A JSON FILE

```
{
  "RootSignature": {
    "Flags": "IRRootSignatureFlagNone",
    "NumParameters": 3,
    "NumStaticSamplers": 1,
    "Parameters": [
      {
        "DescriptorTable": {
          "DescriptorRanges": [
            {
              "BaseShaderRegister": 0,
              "Flags": "IRDescriptorRangeFlagDataStatic",
              "NumDescriptors": 1,
              "OffsetInDescriptorsFromTableStart": 0,
              "RangeType": "IRDescriptorRangeTypeSRV",
              "RegisterSpace": 0
            }
          ],
          "NumDescriptorRanges": 1
        },
        "ParameterType": "IRRootParameterTypeDescriptorTable",
        "ShaderVisibility": "IRShaderVisibilityPixel"
      },
      {
```

```
        "Descriptor": {
          "Flags": "IRRootDescriptorFlagNone",
          "RegisterSpace": 2,
          "ShaderRegister": 0
        },
        "ParameterType": "IRRootParameterTypeCBV",
        "ShaderVisibility": "IRShaderVisibilityPixel"
      },
      {
        "Constants": {
          "Num32BitValues": 4,
          "RegisterSpace": 2,
          "ShaderRegister": 1
        },
        "ParameterType": "IRRootParameterType32BitConstants",
        "ShaderVisibility": "IRShaderVisibilityAll"
      }
    ],
    "StaticSamplers": [
      {
        "AddressU": "IRTextureAddressModeWrap",
        "AddressV": "IRTextureAddressModeWrap",
        "AddressW": "IRTextureAddressModeWrap",
        "BorderColor": "IRStaticBorderColorOpaqueBlack",
        "ComparisonFunc": "IRComparisonFunctionNever",
        "Filter": "IRFilterMinMagMipLinear",
        "MaxAnisotropy": 0,
        "MaxLOD": 3.4028234663852886e+38,
        "MinLOD": 0,
        "MipLODBias": 0,
        "RegisterSpace": 0,
        "ShaderRegister": 0,
        "ShaderVisibility": "IRShaderVisibilityPixel"
      }
    ]
  },
  "version": "IRRootSignatureVersion_1_1"
}
```

C++ BUMP ALLOCATOR

This snippet demonstrates a simple C++ bump allocator, implemented using **metal-cpp**. Note: this allocator is not thread safe. For multithreading encoding, create one of these instances per thread per frame.

```
#ifndef BUMPALLOCATOR_HPP
#define BUMPALLOCATOR_HPP

#include <Metal/Metal.hpp>
#include <tuple>
#include <cassert>
#include <cstdint>

namespace mem
{
```

```
constexpr uint64_t alignUp(uint64_t n, uint64_t alignment)
{
    return (n + alignment - 1) & ~(alignment - 1);
}
}

/// This allocator is not thread safe. For multithreading encoding,
/// create one of these instances per thread per frame.
class BumpAllocator
{
public:
    BumpAllocator(MTL::Device* pDevice,
                  size_t capacityInBytes,
                  MTL::ResourceOptions resourceOptions)
    {
        assert(resourceOptions != MTL::ResourceStorageModePrivate);
        _offset = 0;
        _capacity = capacityInBytes;
        _pBuffer = pDevice->newBuffer(capacityInBytes, resourceOptions);
        _contents = (uint8_t*)_pBuffer->contents();
    }

    ~BumpAllocator()
    {
        _pBuffer->release();
    }

    // Disable copy and move constructors and assignment operators

    void reset() { _offset = 0; }

    template< typename T >
    std::pair<T*, uint64_t> addAllocation(uint64_t count=1) noexcept
    {
        // Shader converter requires an alignment of 8-bytes:
        uint64_t allocSize = mem::alignUp(sizeof(T) * count, 8);

        // If you hit this assert, the allocation data doesn't fit in
        // the amount estimated.
        assert((_offset + allocSize) <= _capacity);

        T* dataPtr          = reinterpret_cast<T*>(_contents + _offset);
        uint64_t dataOffset = _offset;

        _offset += allocSize;

        return { dataPtr, dataOffset };
    }

    MTL::Buffer* baseBuffer() const noexcept
    {
        return _pBuffer;
    }

private:
```

```
    MTL::Buffer* _pBuffer;
    uint64_t _offset;
    uint64_t _capacity;
    uint8_t* _contents;
};

#endif // BUMPALLOCATOR_HPP
```

Complete code samples

All code samples can be found at the Metal sample code page. These samples are complete programs you can use as a starting point for your next project, or just to try out Metal shader converter.

TESSELLATION, GEOMETRY, AND INSTANCING PIPELINES

This sample builds on the "Learn Metal with C++" code sample to add a grass floor to the scene via geometry and tessellation pipeline emulation. The UI allows you to select across the different pipelines available. This sample requires macOS 14 or later.

RAY QUERY

This sample shows how to build a compute kernel that performs ray query operations to find intersections against an acceleration structure containing two triangle instances. This sample requires macOS 13 or later.

RAY TRACING WITH INTERSECTION FUNCTION BUFFER

This sample demonstrates how to convert a DXR ray tracing pipeline to metal, and then emulate a DXR shader table with Metal intersection function buffer if running on macOS 26 or later on Apple M3 or later. Otherwise, this sample falls back to using Metal visible function table.

FUNCTION CONSTANTS AND FRAMEBUFFER FETCH

This sample shows how to adopt Metal-language features in HLSL by demonstrating programmable blending utilizing framebuffer fetch. It also shows how to specialize your HLSL pipelines with function constants.

Code snippets in this document are Copyright (C) 2023-2025 Apple Inc. licensed under the Apache License, Version 2.0.